Project - Low Level Design

Legal Document QA System

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Jogendar Das Bairagi	EN22ME304045
2.	Roshan Bankar	EN22EL301072
3.	Vivek Gangrade	EN22CS3011098
4.	Maahi Maheshwari	EN22ME304057
5.	Shubhangi Prajapat	EN22ME304094

Group Name : **04D10**

Project Number : DO-17

Industry Mentor Name :

University Mentor Name :

Academic Year : 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. **References**

1. Introduction

1.1. Scope of the document

This document provides a detailed low-level design of the AI-powered Legal Document QA System. It details the internal logic, component interactions, and the specific configurations of the LLM and Vector Database layers.

1.2. Intended Audience

- Technical Reviewers and Evaluators.
- LLD evaluators, project stakeholders.
- Development Team.

1.3. System Overview

The system allows users to upload complex legal PDF documents and query them using natural language. It leverages **Llama 3.1** via the **Groq LPU** for high-speed inference and **FAISS** for efficient similarity search, ensuring that responses are grounded strictly in the provided context to prevent hallucinations.

2. System Design

2.1. Application Design

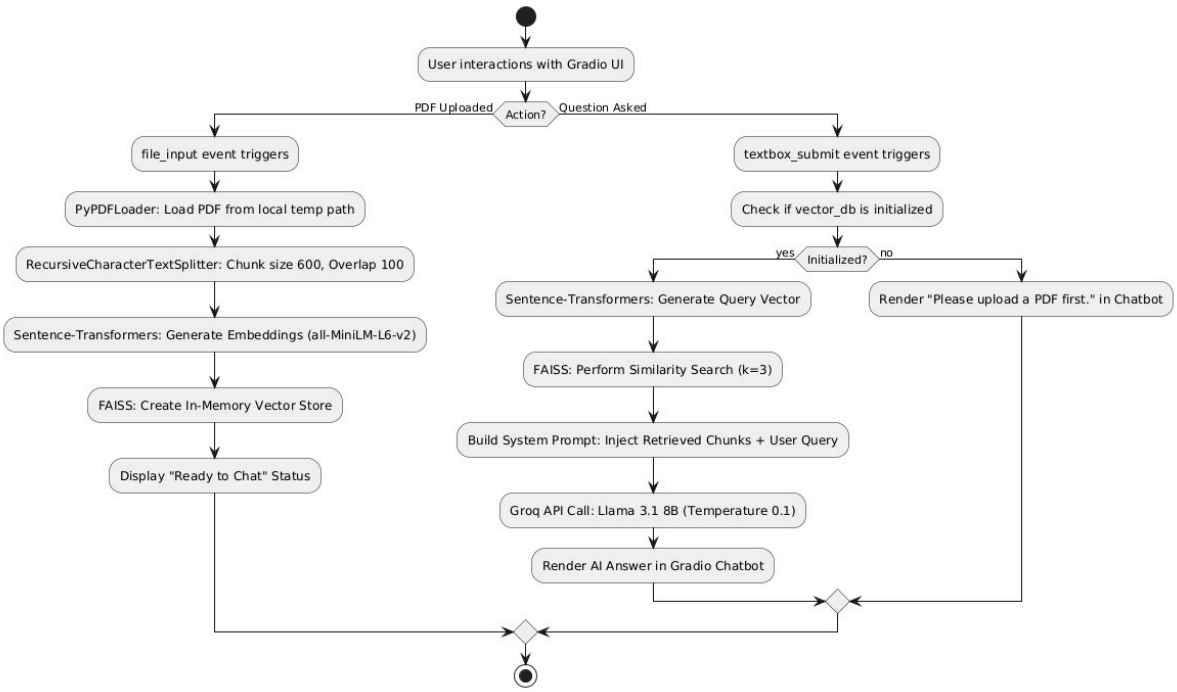
The application follows a Modular Monolith pattern on the backend, using **LangChain** as the orchestration framework.

- **Frontend Interface Layer:** Built with **Gradio**, providing an event-driven UI for file uploads and chat.
- **Service Layer:** Python-based logic handling document chunking and prompt engineering.
- **Data Layer:** A local **FAISS** vector store that holds document embeddings in memory.

2.2. Process Flow

The process is divided into two phases:

1. **Ingestion:** PDF → Text Extraction → Recursive Chunking → Vectorization.
2. **Retrieval:** User Query → Semantic Search → Context Injection → LLM Response.



2.3. Information Flow

1. **Input:** Multipart/form-data (PDF) and String-based user queries.
2. **Transformation:** Text is transformed into 384-dimensional dense vectors using the all-MiniLM-L6-v2 model.
3. **Output:** JSON-based completion objects from the Groq API, rendered as Markdown in the UI.

2.4. Components Design

Component	Technology	Responsibility
Document Loader	PyPDFLoader	Parsing PDF structure and extracting raw text.
Text Splitter	RecursiveCharacterTextSplitter	Breaking text into 600-character chunks with 100-character overlap.
Embeddings	HuggingFaceEmbeddings	Generating semantic signatures for text chunks.
Vector Store	FAISS	Performing K-Nearest Neighbor (KNN) search for relevant context.
LLM Inference	Groq	Generating natural language answers based on retrieved context.

2.5. Key Design Considerations

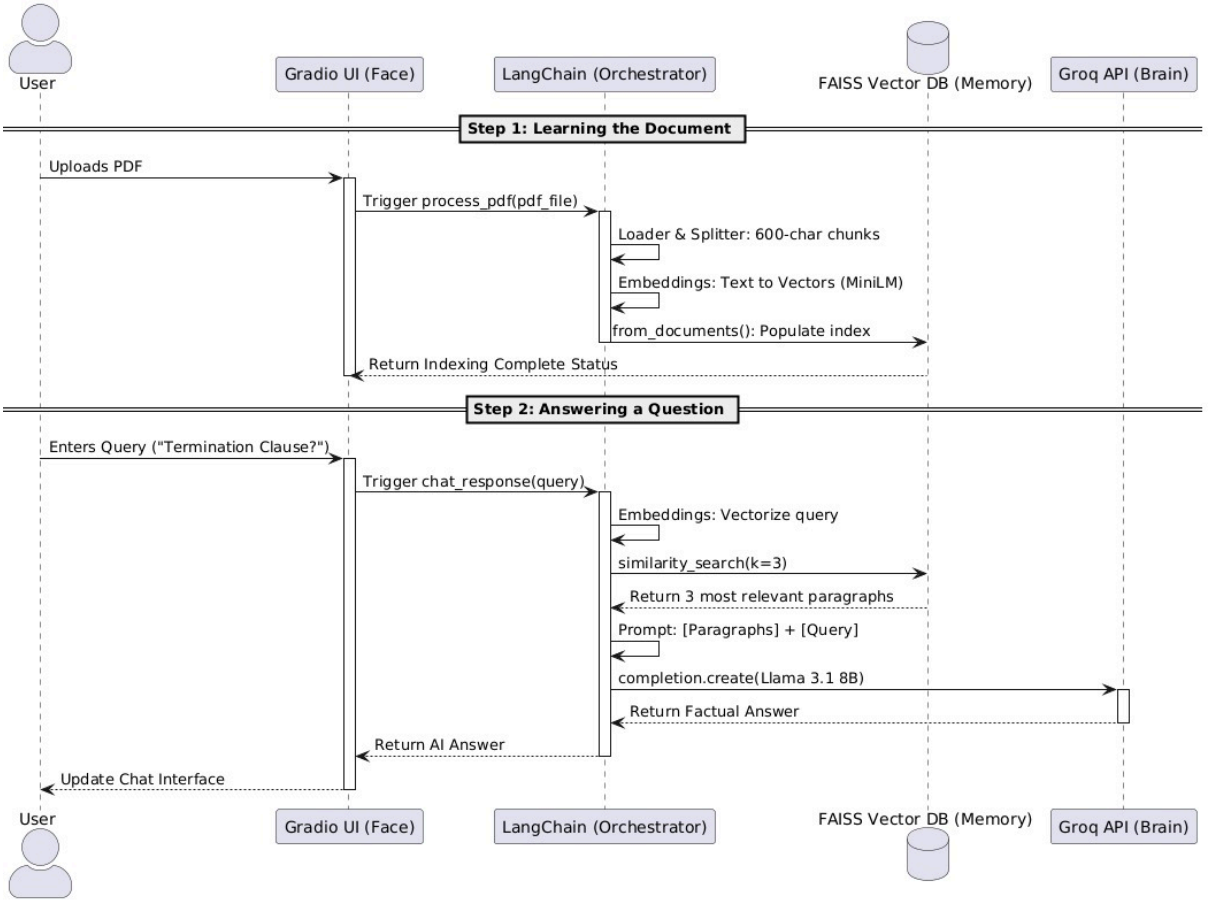
- **Chunk Overlap (100 chars):** Critical for legal documents to ensure that definitions or clauses split at boundaries remain understandable.
- **Temperature (0.1):** Set to low to ensure the model remains deterministic and "greedy," choosing the most probable factual word over creative variations.

- **Grounding:** The system prompt explicitly restricts the LLM to answer *only* from the provided context.

2.6. API Catalogue

- **FAISS.from_documents()**: Internal API to build the vector index.
- **vector_db.similarity_search()**: Internal API to retrieve relevant document chunks.
- **client.chat.completions.create()**: External API call to the Groq cloud for LLM inference.

4. Sequence Diagram



5. References

- LangChain Documentation: https://python.langchain.com
- Groq API Reference: https://console.groq.com/docs
- FAISS (Facebook AI Similarity Search): https://github.com/facebookresearch/faiss